

Kreacijski paterni

Uvod

U razvoju softvera, kreacijski paterni (Creational Design Patterns) su provjereni šabloni koji se bave procesom kreiranja objekata. Navikli smo objekte kreirati direktno pomoću ključne riječi `new` (npr. `Torta mojaTorta = new Torta();`). Međutim, u velikim i kompleksnim sistemima, ovakav pristup stvara problem koji zovemo tesna spregnutost (tight coupling). Ako kontroleri ili menadžeri direktno instanciraju konkretne klase, sistem postaje nefleksibilan. Ako se promijeni način pravljenja objekta ili se uvede nova podklasa, kod se mora mijenjati na puno mjesta.

Kreacijski paterni rješavaju ovaj problem tako što apstrahuju i izoluju logiku instanciranja. Oni odgovaraju na pitanje: *Kako stvoriti objekat, a da aplikacija ne zavisi od njegovih konkretnih klasa, složenosti njegove konstrukcije ili načina na koji se njime upravlja u memoriji?*

Tri glavna cilja kreacijskih paterni

1. Enkapsulacija (Skrivanje) logike: Detalji o tome *kako* se objekat pravi (koji konstruktori se pozivaju, koje parametre vuče) sklanjaju se iz glavnog koda (npr. iz kontrolera).
2. Fleksibilnost i proširivost: Omogućavaju sistemu da odluči *koji* objekat treba kreirati u pokretu (runtime-u), bez prepravljjanja postojeće arhitekture.
3. Kontrola nad resursima: Kontrolišu *koliko* objekata se stvara i *na koji način* (npr. da li nam treba samo jedna zajednička instanca ili želimo brzo kopirati postojeću).

1. Singleton

Singleton osigurava da klasa ima samo jednu instancu u cijelom sistemu i pruža globalnu pristupnu tačku toj instanci. Koristi se kada dijeljeni resurs (poput baze podataka ili eksternog servisa) zahtijeva centralizovanu kontrolu.

- Implementacija u našem sistemu:
 - Primjenjuje se: na `DataContext` klasa (povezana sa bazom podataka) ili eksterni Email servis koji se koristi za slanje asinhronih obavještenja o statusu narudžbe
 - Logika implementacije: Umjesto da svaki kontroler (`TortaController`, `NarudžbaController`) kreira novu instancu baze podataka kroz `new DataContext()`, `DataContext` treba imati privatni konstruktor i statičku metodu `Instance` (ili se registruje kao *Singleton lifecycle* unutar *Dependency Injection* kontejnera u web aplikaciji)

- Prednost: Smanjuje se potrošnja memorije i sprječava se konflikt nad bazom kada više korisnika istovremeno pokušava rezervirati isti termin na kalendaru

2. Prototype

Omogućava kreiranje novih objekata kopiranjem (kloniranjem) postojećeg objekta (prototipa) umjesto kreiranjem "od nule". Koristi se kada je proces instanciranja objekta skup ili komplikovan.

- Implementacija u našem sistemu:
 - Primjenjuje se: na klasi Torta
 - Logika implementacije: Obzirom da u našoj specifikaciji stoji da su bento torte standardizovane veličine, možemo definisati bazne "prototipe" torti (npr. *nutella bento torta*, *raffaelo bento torta*) koji već imaju predefinisane bazične okuse ili oblike. Kada korisnik otvori formu za personalizaciju, sistem ne pravi potpuno novi objekat, već klonira taj bazni prototip. Korisnik zatim na toj kopiji vrši izmjene – dodaje tekst, dekoraciju (npr. JESTIVE_PERLICE) ili referentnu sliku
 - Prednost: Značajno ubrzava rad sistema, jer aplikacija ne mora svaki put iznova učitavati sve podrazumijevane koeficijente i parametre iz baze prilikom inicijalizacije forme za personalizaciju

3. Factory Method

Definira interfejs za kreiranje objekata, ali prepušta podklasama da odluče koju će klasu instancirati. Omogućava klasi da delegira logiku instanciranja podklasama.

- Implementacija u našem sistemu:
 - Primjenjuje se: između TortaController-a i domenske klase Torta
 - Logika implementacije: Kreiramo apstraktnu klasu ili interfejs BentoFactory sa metodom NapraviTortu(). Zatim kreiramo konkretne fabrike: StandardnaTortaFactory (koja vraća običnu bento tortu) i PremiumTortaFactory (koja vraća kompleksniju tortu sa npr. kompleksnijim dekoracijama i višim koeficijentom cijene). TortaController samo poziva NapraviTortu(), potpuno nesvjestan koja tačno podklasa torte se kreira
 - Prednost: Savršeno podržava proširivost. Ako sutra slastičarna odluči uvesti novu kategoriju (npr. *veganske bento torte* ili *sezonske torte*), samo ćemo dodati VeganskaTortaFactory, bez ikakve potrebe da mijenjamo postojeći kod u TortaController-u ili bazi podataka

4. Abstract Factory

Pruž a interfejs za kreiranje čitavih porodica povezanih ili zavisnih objekata bez specificiranja njihovih konkretnih klasa.

- Implementacija u našem sistemu:
 - Primjenjuje se: za kreiranje porodica objekata koji zavise od Načina preuzimanja narudžbe (LICNO_PREUZIMANJE ili DOSTAVA).
 - Logika implementacije: Narudžba za lično preuzimanje i narudžba za dostavu povlače sa sobom različite povezane objekte. Možemo kreirati AbstractNarudžbaFactory:
 - LicnoPreuzimanjeFactory će kreirati objekat Narudžba (gdje je cijena dostave 0) i pripadajući objekat Obavještenje ("Vaša torta vas čeka u poslovnici")
 - DostavaFactory će kreirati objekat Narudžba (sa uvećanom cijenom), objekat Dostava (sa adresom i brojem telefona) i objekat Obavještenje ("Kurir je preuzeo vašu tortu")
 - Prednost: Osigurava konzistentnost, nemoguće je da kreiramo narudžbu za dostavu, a da sistem zaboravi kreirati prateći entitet Dostava ili da pošalje pogrešan tip obavještenja korisniku.

5. Builder patern

Razdvaja konstrukciju složenog objekta od njegove reprezentacije, omogućavajući da se isti proces konstrukcije iskoristi za kreiranje različitih reprezentacija. Koristi se za objekte koji se grade "korak po korak".

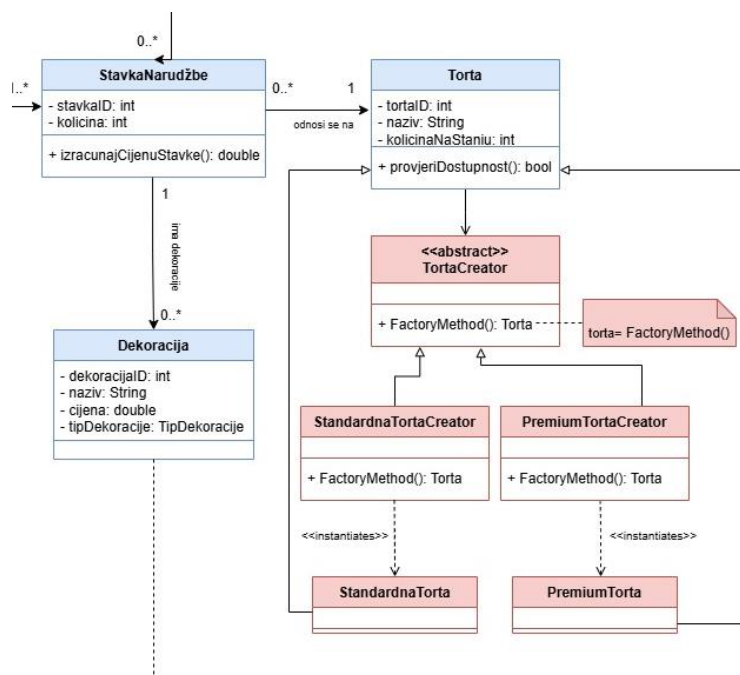
- Implementacija u našem sistemu:
 - Primjenjuje se: na kompleksnoj klasi Narudžba koja u sebi agregira Korisnik-a, listu StavkaNarudžbe, vezu prema Dostava, status i proračun ukupne cijene
 - Logika implementacije: Kreira se klasa NarudžbaBuilder. Unutar NarudžbaController-a, tokom korisničkog prolaska kroz korake (dodavanje torte u korpu, unos teksta, odabir datuma na kalendaru, odabir dostave), builder postepeno puni objekat preko metoda: .PostaviKorisnika(korisnik) → .DodajStavke(listaStavki) → .DodajDostavu(podaciODostavi) → .IzračunajCijenuISloženost(). Tek na samom kraju, kada korisnik klikne "Potvrdi", poziva se metoda .Build() koja vraća potpuno sklopljen i validan objekat Narudžba
 - Prednost: Rješava problem parcijalno kreiranih objekata. Kako naša specifikacija navodi u alternativnim tokovima, korisnik može odustati u bilo kojem koraku, korištenjem Buildera polovični podaci ostaju samo u builderu u memoriji i uopšte ne opterećuju domenski model niti se šalju u DataContext sve dok proces nije u potpunosti uspješno završen

Novi klasni dijagram sa dodana dva kreacijska paterna

U skladu sa zahtjevima i specifikacijom projekta, odabrana su i implementirana dva kreacijska paterna: *Factory Method* i *Builder*.

- **Factory Method (Kreiranje Torte)**

- Šta je dodano: Uvedena je apstraktna klasa kreatora *TortaCreator* sa polimornom metodom + *factoryMethod()*: *Torta*, te dvije konkretne podklase kreatora: *StandardnaTortaCreator* i *PremiumTortaCreator*.
- Zašto: Iako se u pozadini sve bento torte perzistiraju u istu domensku klasu *Torta* radi jednostavnosti baze podataka, poslovna logika njihove inicijalizacije se drastično razlikuje. *StandardnaTortaCreator* postavlja bazične koeficijente i standardne parametre, dok *PremiumTortaCreator* izoluje kompleksniju logiku koja uključuje veće koeficijente složenosti i pripremu objekta za ručnu procjenu dizajna od strane zaposlenika.
- Kako je urađeno: Konkretne fabrike nasljeđuju apstraktni *TortaCreator* i preklapaju metodu *factoryMethod()*, koja kroz usmjerenu asocijaciju direktno instancira (<<instantiates>>) odgovarajuće konfigurisan objekat klase *Torta*.



- **Builder patern (Konstrukcija Narudžbe)**

- Šta je dodano: Uveden je interfejs *INarudzbaBuilder*, njegova konkretna implementacija *SistemskaNarudzbaBuilder*, dok je postojeća klasa *NarudzbaController* preuzela arhitektonsku ulogu upravljača (Director).

- Zašto: Klasa Narudzba predstavlja izuzetno složen (kompleksan) objekat u sistemu jer se sastoji od niza drugih povezanih komponenti: podataka o kupcu (Korisnik), liste stavki (StavkaNarudzbe) i informacija o transportu (Dostava). Direktno kreiranje ovako složenog objekta unutar kontrolera narušilo bi principe čistog koda.
- Kako je urađeno: Proces gradnje je podijeljen u jasne korake kroz metode interfejsa: buildKorisnik(), buildStavke() i buildDostava(), dok metoda getResult() vraća kompletiran proizvod. NarudzbaController upravlja procesom preko asocijacije sa interfejsom (koristeći metodu + Construct()), čime je potpuno izolovan od samog procesa sklapanja i unutrašnje strukture objekta Narudzba.

